JavaScript Promises — Complete Notes

1. What Is a Promise?

Definition: A Promise is a JavaScript object that represents the eventual success or failure of an asynchronous operation.

A Promise allows us to handle operations that may take time, such as:

- Fetching data from an API.
- Reading data from a server.
- Processing payments.
- Loading user details.
- Saving data into a database.

Real-life example:

Imagine you order food online.

1. You place the order.
2. The restaurant starts preparing it.
3. The order is either delivered successfully or cancelled.

While waiting, you do not know the final result. A Promise represents this pending operation and its eventual outcome.

2. Why Do We Need Promises?

Before Promises, JavaScript developers often used callbacks to handle asynchronous operations.

When many dependent callbacks are nested inside one another, the code can become difficult to read. This problem is called Callback Hell.

Promises help us:

1. Avoid deeply nested callbacks through chaining.
2. Handle success and failure in a structured way.
3. Execute dependent asynchronous operations.
4. Write more readable asynchronous code.
5. Centralize error handling using `.catch()`.

Important: Promises do not automatically make code execute faster. They provide a better way to manage asynchronous results.

3. States of a Promise

A Promise has three states:

State	Meaning
Pending	The operation has not settled yet.
Fulfilled	The operation completed successfully.
Rejected	The operation failed.

A Promise starts as pending and eventually becomes fulfilled or rejected.

Once a Promise is settled, its state cannot change again.

Example:

```
const promise = new Promise((resolve, reject) => {
    resolve("Task completed successfully");
});

console.log(promise);
```

The Promise becomes fulfilled because `resolve()` is called.

4. Creating a Promise

We create a Promise using the `Promise` constructor.

Syntax:

```
const promise = new Promise((resolve, reject) => {
    // Perform an operation

    resolve("Success message");
    // Or use reject("Error message");
});
```

Here:

- `Promise` is the built-in JavaScript constructor.
- `resolve()` fulfills the Promise with a value.
- `reject()` rejects the Promise with a reason.
- The function passed to the constructor is called the executor function.

The executor function runs immediately when the Promise is created.

Example 1: Successful Promise

```
const myPromise = new Promise((resolve, reject) => {
    resolve("Registration successful!");
```

```
});

myPromise.then((result) => {
    console.log(result);
});
```

Output:

```
Registration successful!
```

Example 2: Rejected Promise

```
const myPromise = new Promise((resolve, reject) => {
    reject("Registration failed!");
});

myPromise.catch((error) => {
    console.log(error);
});
```

Output:

```
Registration failed!
```

5. The setTimeout() Example

We commonly use `setTimeout()` to simulate an operation that takes time.

```
const myPromise = new Promise((resolve, reject) => {
    setTimeout(() => {
        resolve("Data loaded successfully!");
    }, 2000);
});

myPromise.then((result) => {
    console.log(result);
});

console.log("Loading...");
```

Output:

```
Loading...
Data loaded successfully!
```

The first message appears immediately. The second appears after approximately two seconds.

Why?

The Promise executor runs immediately, but the callback inside `setTimeout()` runs later. The `.then()` handler runs after the Promise is fulfilled.

6. The then() Method

The `.then()` method handles a fulfilled Promise. It can also accept a second callback to handle rejection, although separate `.catch()` handling is often clearer.

Syntax:

```
promise.then(
    (result) => {
        // Handle success
    },
    (error) => {
        // Handle failure
    }
);
```

Example:

```
const promise = Promise.resolve("Login successful");

promise.then((result) => {
    console.log(result);
});
```

Output:

```
Login successful
```

Important: `.then()` returns a new Promise, which allows Promise chaining.

7. The catch() Method

The `.catch()` method handles a rejected Promise or an error thrown by an earlier handler in a Promise chain.

Example:

```
const promise = Promise.reject("Invalid password");

promise.catch((error) => {
    console.log(error);
});
```

Output:

```
Invalid password
```

Handling errors in a chain

```
Promise.resolve()
    .then(() => {
        throw new Error("Something went wrong");
    })
    .catch((error) => {
        console.log(error.message);
    });
```

Output:

```
Something went wrong
```

The thrown error rejects the Promise returned by `.then()`, and `.catch()` handles it.

8. The finally() Method

The `.finally()` method executes after a Promise settles, whether it is fulfilled or rejected.

It is useful for cleanup tasks such as hiding a loading indicator.

Example:

```
const promise = Promise.resolve("Data received");

promise
    .then((result) => {
        console.log(result);
    })
    .catch((error) => {
        console.log(error);
    })
    .finally(() => {
```

```
        console.log("Operation finished");
    });
```

Output:

```
Data received
Operation finished
```

Important points:

- `finally()` does not receive the success value or rejection reason as an argument.
- It normally preserves the original outcome.
- If `finally()` throws an error or returns a rejected Promise, the resulting Promise becomes rejected.

9. Promise Chaining

Promise chaining means executing multiple `.then()` handlers in sequence.

The value returned by one handler becomes the input to the next handler.

Example:

```
Promise.resolve(10)
    .then((number) => {
        console.log(number);
        return number * 2;
    })
    .then((number) => {
        console.log(number);
        return number + 5;
    })
    .then((number) => {
        console.log(number);
    });
```

Output:

```
10
20
25
```

How does it work?

1. The first handler receives `10`.
2. It returns `20`.
3. The second handler receives `20` and returns `25`.

4. The third handler receives `25`.

Important chaining rule

Always return a value when the next `.then()` needs it.

```
Promise.resolve(5)
    .then((number) => {
        return number * 2;
    })
    .then((result) => {
        console.log(result);
    });
```

Output: `10`

If you omit `return`, the next handler normally receives `undefined`.

10. Returning a Promise Inside then()

A `.then()` handler can return another Promise. The next handler waits for that Promise to settle.

```
function getUser() {
    return new Promise((resolve) => {
        setTimeout(() => {
            resolve("Harsh");
        }, 1000);
    });
}

Promise.resolve()
    .then(() => {
        return getUser();
    })
    .then((name) => {
        console.log("User:", name);
    });
```

Output after approximately one second:

```
User: Harsh
```

Returning `getUser()` is important because it connects that asynchronous operation to the chain.

11. Promise.resolve()

`Promise.resolve(value)` creates a fulfilled Promise with the supplied value. If the value is a Promise or thenable, the returned Promise adopts its eventual state.

```
Promise.resolve("Success")
    .then((result) => {
        console.log(result);
    });
```

Output:

```
Success
```

This is useful when you want to convert a normal value into a Promise.

12. Promise.reject()

`Promise.reject(reason)` creates a rejected Promise.

```
Promise.reject("Access denied")
    .catch((error) => {
        console.log(error);
    });
```

Output:

```
Access denied
```

This is useful when you need to produce a rejected Promise deliberately.

13. Promise.all()

`Promise.all()` accepts an iterable of values and Promises and returns one Promise.

- It fulfills when all inputs fulfill.
- Its result is an array of values in the original input order.
- It rejects as soon as an input rejects.
- It does not automatically cancel the remaining operations.

Example:

```
const p1 = Promise.resolve("User");
const p2 = Promise.resolve("Courses");
const p3 = Promise.resolve("Results");
```

```
Promise.all([p1, p2, p3])
    .then((results) => {
        console.log(results);
    })
    .catch((error) => {
        console.log(error);
    });
```

Output:

```
[ 'User', 'Courses', 'Results' ]
```

When should we use Promise.all()?

Use it when several independent operations can run concurrently and you need all of their results.

For example, loading a user's profile, notifications, and course list at the same time.

14. Promise.allSettled()

`Promise.allSettled()` waits until every input Promise settles.

It returns an array describing the outcome of each input, regardless of success or failure.

Example:

```
const p1 = Promise.resolve("Payment successful");
const p2 = Promise.reject("Payment failed");
const p3 = Promise.resolve("Bill generated");

Promise.allSettled([p1, p2, p3])
    .then((results) => {
        console.log(results);
    });
```

Output:

```
[
    { status: "fulfilled", value: "Payment successful" },
    { status: "rejected", reason: "Payment failed" },
    { status: "fulfilled", value: "Bill generated" }
]
```

The exact formatting shown by the console may differ.

When should we use it?

Use it when every operation matters, even if some fail. For example, sending notifications to several users and recording which deliveries succeeded or failed.

15. Promise.race()

`Promise.race()` settles according to the first input Promise that settles, whether that outcome is fulfillment or rejection.

Example:

```
const p1 = new Promise((resolve) => {
    setTimeout(() => resolve("Task 1 completed"), 2000);
});

const p2 = new Promise((resolve) => {
    setTimeout(() => resolve("Task 2 completed"), 1000);
});

Promise.race([p1, p2])
    .then((result) => {
        console.log(result);
    });
```

Output:

```
Task 2 completed
```

When should we use it?

Use it when you need the first settled result, such as implementing a timeout race.

Important: `Promise.race()` does not cancel the slower operations.

16. Promise.any()

`Promise.any()` fulfills as soon as any input Promise fulfills.

It ignores rejections while waiting for a successful result. If every input rejects, it rejects with an `AggregateError`.

Example:

```
const p1 = Promise.reject("Server 1 failed");
const p2 = Promise.resolve("Server 2 successful");
const p3 = Promise.resolve("Server 3 successful");
```

```
Promise.any([p1, p2, p3])
    .then((result) => {
        console.log(result);
    })
    .catch((error) => {
        console.log(error);
    });
```

Output:

```
Server 2 successful
```

When should we use it?

Use it when any successful result is enough, such as requesting the same resource from multiple alternative servers.

If all inputs reject, the `AggregateError.errors` property contains their rejection reasons.

17. Difference Between Promise.all(), allSettled(), race(), and any()

Method	When does it settle?	Result
`Promise.all()`	All fulfill, or one rejects	Array of values, or first rejection
`Promise.allSettled()`	All inputs settle	Array of outcome objects
`Promise.race()`	First input settles	First fulfillment value or rejection
`Promise.any()`	First input fulfills, or all reject	First fulfillment value, or `AggregateError`

Easy memory trick:

- `all()` = All must succeed.
- `allSettled()` = Wait for all outcomes.
- `race()` = First settled outcome wins.
- `any()` = First success wins.

18. Sequential Execution Using Promises

Sometimes the next operation depends on the result of the previous operation. In that case, use Promise chaining.

Example: Order processing

```
function getOrder() {
    return Promise.resolve("Order placed");
}
```

```
function processPayment() {
    return Promise.resolve("Payment completed");
}

function generateBill() {
    return Promise.resolve("Bill generated");
}

getOrder()
    .then((result) => {
        console.log(result);
        return processPayment();
    })
    .then((result) => {
        console.log(result);
        return generateBill();
    })
    .then((result) => {
        console.log(result);
    })
    .catch((error) => {
        console.log("Error:", error);
    });
```

Output:

```
Order placed
Payment completed
Bill generated
```

The returned Promise connects each operation to the next step.

19. Parallel Execution Using Promises

If operations are independent, start them together and use `Promise.all()`.

```
function getUser() {
    return new Promise((resolve) => {
        setTimeout(() => resolve("User loaded"), 1000);
    });
}

function getCourses() {
    return new Promise((resolve) => {
        setTimeout(() => resolve("Courses loaded"), 1000);
    });
}

Promise.all([getUser(), getCourses()])
```

```
    .then((results) => {
        console.log(results);
    });
```

Output after approximately one second:

```
[ 'User loaded', 'Courses loaded' ]
```

The operations run concurrently rather than waiting for one another.

20. Error Handling With Multiple then() Handlers

A rejection or thrown error travels down the chain until a rejection handler handles it.

```
Promise.resolve()
    .then(() => {
        console.log("Step 1");
        throw new Error("Step 2 failed");
    })
    .then(() => {
        console.log("Step 3");
    })
    .catch((error) => {
        console.log(error.message);
    })
    .then(() => {
        console.log("Process finished");
    });
```

Output:

```
Step 1
Step 2 failed
Process finished
```

The second success handler is skipped because the previous handler throws an error. After `.catch()` handles it normally, the chain continues with the next `.then()`.

21. Promise With async/await

`async/await` provides a readable way to work with Promises.

- An `async` function always returns a Promise.
- `await` waits for a Promise's outcome inside an async function.
- If the awaited Promise rejects, the error can be handled using `try...catch`.

Example:

```
function getData() {
    return new Promise((resolve) => {
        setTimeout(() => {
            resolve("Data received successfully");
        }, 1000);
    });
}

async function displayData() {
    try {
        const result = await getData();
        console.log(result);
    } catch (error) {
        console.log("Error:", error);
    }
}

displayData();
```

Output after approximately one second:

```
Data received successfully
```

22. Common Mistakes With Promises

Mistake 1: Forgetting to return a value

```
Promise.resolve(10)
    .then((number) => {
        number * 2;
    })
    .then((result) => {
        console.log(result);
    });
```

Output: `undefined`

Correct approach:

```
Promise.resolve(10)
    .then((number) => {
        return number * 2;
    })
    .then((result) => {
```

```
        console.log(result);
    });
```

Output: `20`

Mistake 2: Forgetting to return a Promise

If you start an asynchronous operation inside `.then()` but do not return its Promise, the chain does not wait for it.

Mistake 3: Thinking Promise creation is asynchronous

The executor function passed to `new Promise()` runs immediately. Promise handlers such as `.then()` run asynchronously as microtasks.

Mistake 4: Thinking Promise.all() cancels other operations

`Promise.all()` rejects when an input rejects, but it does not automatically cancel the remaining work.

Mistake 5: Forgetting error handling

Use `.catch()` or `try...catch` with `await` so failures can be handled properly.

23. Promise Execution Order

Consider the following example:

```
console.log("Start");

Promise.resolve().then(() => {
    console.log("Promise");
});

console.log("End");
```

Output:

```
Start
End
Promise
```

Why?

1. `"Start"` is printed synchronously.
2. The Promise executor, if there is one, runs immediately; here `Promise.resolve()` creates an already-fulfilled Promise.
3. The `.then()` callback is scheduled as a microtask.

4. `"End"` is printed synchronously.
5. The microtask runs and prints `"Promise"`.

Promise handlers run as microtasks, which normally execute after the current synchronous JavaScript work finishes.

24. Important Promise Rules

1. A Promise can be pending, fulfilled, or rejected.
2. A settled Promise cannot change its state again.
3. `.then()` handles fulfillment and can also handle rejection if a second handler is supplied.
4. `.catch()` handles rejection and errors propagated through a chain.
5. `.finally()` runs after settlement.
6. `.then()`, `.catch()`, and `.finally()` each return a new Promise.
7. Return values and returned Promises determine how a chain continues.
8. `Promise.all()` is useful for concurrent operations that all need to succeed.
9. `Promise.allSettled()` records every outcome.
10. `Promise.race()` chooses the first settled outcome.
11. `Promise.any()` chooses the first successful outcome.
12. `async/await` is syntax built around Promises.

25. Final Summary

A Promise is a JavaScript object used to represent the eventual outcome of an operation.

- **Create:** `new Promise()`
- **Fulfill:** `resolve()`
- **Reject:** `reject()`
- **Handle success:** `.then()`
- **Handle errors:** `.catch()`
- **Run cleanup:** `.finally()`
- **Wait for all successes:** `Promise.all()`
- **Wait for all outcomes:** `Promise.allSettled()`
- **Get the first settled result:** `Promise.race()`
- **Get the first successful result:** `Promise.any()`
- **Write readable asynchronous code:** `async/await`

Remember: Promises help organize asynchronous operations and error handling. Use chaining for dependent tasks and methods such as `Promise.all()` for independent tasks that can run concurrently.